

Simulation Design of Load Balancing Optimization for Cloud Computing Data Stream Storage Based on Big Data Algorithms

Jing Gao

Shandong Polytechnic College, Jining, Shandong, 272067 China
397757915@qq.com

Abstract—Optimizing the storage load balancing of data streams in cloud computing can improve the uniformity of data stream storage distribution within the network. The traditional method's processing power is only limited to the window range that a certain operator can handle at the node. When the data gradually increases, the processing power is insufficient, which can easily lead to data flow congestion. Moreover, it ignores the research on the load distribution of the entire system and migration strategies in dynamic load balancing. Optimizing load balancing of data streams requires evaluating the load situation of data stream storage nodes, transferring high-intensity loads to nodes with lighter loads, and completing the optimization of data stream storage load balancing. To address the shortcomings of existing load balancing methods, this paper proposes a simulation design method for optimizing load balancing of cloud computing data stream storage based on big data algorithms. This method analyzes the historical load data of storage nodes, uses big data algorithms to predict the future load situation of nodes, and dynamically allocates data to corresponding nodes based on this to achieve balanced distribution and optimization of load. The experimental results show that the proposed method has high feasibility, good computational performance, and dynamic load balancing.

Keywords—Big data algorithms, Cloud computing, Data flow, Storage, load balancing

I. INTRODUCTION

In the past 10 years, with the development of internet technology and the rise of multimedia, mobile communication, and the Internet of Things technology, data has shown explosive growth in different fields[1]. Cloud computing is a new computing model supported by virtualization technology as the underlying technology, integrating distributed computing, load balancing, and network storage. It is an inevitable trend and product to meet the current development of IT technology[2]. Considering that the storage process of data streams in cloud computing requires necessary category merging and transmission, people's demand for accuracy and bandwidth in the storage process of data streams is gradually increasing. By using effective data stream storage load balancing mechanisms to store data streams, it has gradually become a hot research topic in the current field of cloud computing. Data stream storage faces challenges such as huge data scale, high processing speed requirements, and high storage costs. At the same time, load balancing technology also needs to solve how to achieve load balancing and optimization in the case of limited storage resources[3]. Therefore, studying the load balancing optimization problem of cloud computing data stream storage has important theoretical and practical

value.

Traditional databases are difficult to meet the needs of precise screening, compatibility, and massive storage and extraction of big data, resulting in difficulties for data storage systems to quickly and efficiently transmit and extract data[4]. Due to the persistence and infinity of data flow, data skewing is inevitable over time, and data skewing can lead to uneven load among processing nodes. In the existing cloud computing data stream storage, distributed storage is usually adopted, where data is fragmented and stored on multiple nodes. However, with the continuous expansion of data scale, the problem of load imbalance among storage nodes is gradually becoming prominent. Load imbalance can lead to some nodes being overloaded while others are idle, thereby affecting the performance and efficiency of the entire storage system[5]. Therefore, studying how to achieve load balancing optimization to improve the performance and efficiency of cloud computing data stream storage is currently an urgent problem to be solved.

The existing load balancing methods mainly include static load balancing and dynamic load balancing. Static load balancing achieves preliminary load balancing by pre calculating the load situation of each node and allocating data to the corresponding nodes[6]. However, due to inaccurate pre calculated load conditions, static load balancing often fails to achieve true load balancing. Dynamic load balancing dynamically allocates data to corresponding nodes based on real-time monitoring of node load, in order to achieve real-time load balancing[7]. However, dynamic load balancing requires a significant amount of monitoring and scheduling overhead, and for large-scale data stream storage, the cost of real-time monitoring and scheduling may exceed its optimization effect. To address the shortcomings of existing load balancing methods, this paper proposes a simulation design method for optimizing load balancing of cloud computing data stream storage based on big data algorithms. This method analyzes the historical load data of storage nodes, uses big data algorithms to predict the future load situation of nodes, and dynamically allocates data to corresponding nodes based on this to achieve balanced distribution and optimization of load.

II. METHODOLOGY

A. Load Balancing Optimization Method for Cloud Computing Data Stream Storage

Cloud computing has evolved from grid computing and

distributed evolution. It provides services to users by charging corresponding fees according to their needs. Through virtualization technology, distributed computing resources equipped on the Internet are centrally allocated. End users who use cloud computing services do not need to have solid professional knowledge, but only need to pay attention to the resources they need and rent corresponding resources according to their own service needs[8]. Dynamic load balancing of data flow refers to migrating the computational load of nodes with high load to nodes with low load to achieve data flow load balancing. For low-priority data streams, a multi threshold prediction mechanism is used to evaluate the load situation of data

stream storage nodes to prevent excessive storage task load on a certain node. In order to effectively control the routing of data tuples, the output data flow of nodes is divided into "blocks", and the data blocks are managed through parallel computing entropy in a network parallel computing environment to complete load migration. The dynamic virtual coordinate mechanism transfers the high-intensity storage load to nodes with lighter loads after the data flow storage node load reaches the threshold, achieving the goal of data flow storage load balancing. Figure 1 shows the load balancing scheduling flowchart and fitness function analysis of cloud computing data stream storage.

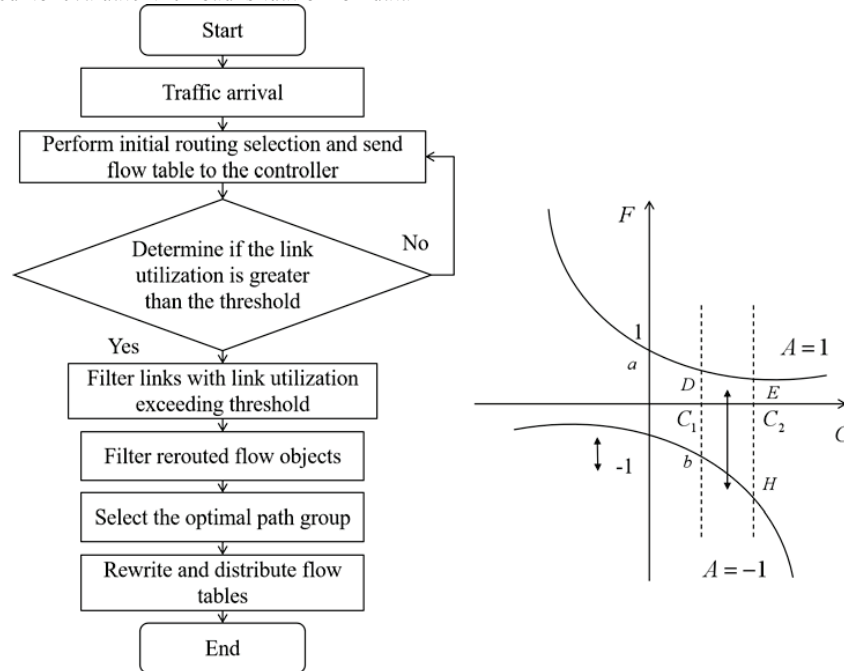


Figure 1 Cloud computing data stream storage load balancing scheduling flowchart and fitness function analysis diagram

After the data flow arrives, it is necessary to select a target node for processing. Firstly, check whether there are idle nodes in the area. When the load of nodes in the area is relatively high, expand to adjacent areas at the same level to ensure the optimal utilization of regional resources[9]. Faced with the demand for high-performance distributed load balancing characteristics in data center networks, the dynamic load balancing algorithm model considers two core principles: using polling to send instruction packets; Considering the user identification code, the same user identification code needs to be prioritized and sent to the same server to avoid complex communication between servers and improve the overall response speed of the data center network system. When the load distribution of cloud storage data is uneven, it is necessary to schedule the elephant flow on the overloaded link. Multiple data streams in the network can dynamically switch in real-time between normal forwarding and discarding, thereby achieving efficient regulation of massive data streams in the network. Set the link utilization threshold to 75%. If the threshold is set too high, scheduling will only begin after the network is excessively congested. Conversely, if the threshold is set too low, it will not be fully utilized in terms of network capacity, and if the computational load is too large, it will affect

network performance. When one or several servers in the dynamic load balancing architecture fail, the dynamic load balancing controller will automatically switch user requests to other available servers, select the best performing server to respond to customer requests within a short delay, reduce user waiting response time, improve system processing capacity, and ensure user access quality, reliability, and efficiency[10].

B. Analysis of Load Balancing Algorithms for Cloud Computing Data Stream Storage

Considering the characteristics of load intensity and minimum transmission granularity in the process of load balancing optimization in cloud computing data stream storage, the fingerprint gradient in the process of data stream load balancing optimization is optimized. The separation of data streams is achieved based on the division of fingerprint gradients, improving the efficiency of data stream storage load balancing and allocation. Adopting a fitness function based on variance partitioning criteria and comprehensive utility values to achieve balanced allocation of data traffic in cloud storage. The link utilization variance of all networks and the link utilization variance corresponding to the links included in the path group in the

repeated paths are represented by B_t and B_r , respectively. The corresponding variance calculation formula is as follows

$$B_t = \frac{\sum_{k=1}^N [\overline{\eta_k(u)} - \eta_k(u)]^2}{N} \quad (1)$$

$$B_r = \frac{\sum_{k=1}^M [\overline{\eta_k(u)} - \eta_k(u)]^2}{M} \quad (2)$$

In the formula, the total number of links in the network topology is represented as N , the total number of links contained in the path group in the repeated path is represented as M , and the average link utilization rate in the k link at u time is represented as $\eta_k(u)$.

Assuming that the cloud computing data stream to be stored is backed up on different storage nodes, the selection of usage strength for each node can be based on the storage load strength $T(x)$ of the data stream and the probability of network transmission bandwidth occupancy $P(x)$. The first moment $E[T(x)]$ of the storage load strength $T(x)$ of the data stream in cloud computing meets the following conditions

$$E[T(x)] = E[P(x)] = \frac{\lambda - E[T(x)]}{\lambda^2 E[T(x)^2] - E[T(x)] - e} \quad (3)$$

Among them, $E[T(x)]$ and $E[T(x)^2]$ represent the expected usage strength $T(x)$ and second-order expectation of the data stream storage load.

Load migration requires a certain amount of resources and time. If the load migration is completed irregularly, it will largely lead to congestion in data flow processing. Therefore, making migration decisions is very important. For smart grid data, there are many types, and different data types contain significant differences in information. For example, alarm data has higher value than conventional data. Therefore, when scheduling, it is necessary to pay attention to the priority of data flow processing tasks. For cases where the utilization of network links exceeds the threshold when there is disturbance in the network, a threshold based variance partitioning standard is used, as shown in equation (4). All link variances contained in the path group are graded based on variance partitioning criteria, and divided into two levels: greater than threshold level and less than threshold level.

$$A = \frac{|B_t - B_r + \omega|}{B_r - B_t + \omega} \quad (4)$$

In the equation, the minimum value is represented as ω , avoiding a denominator of 0. If $B_t < B_r$, then $A = -1$, where the variance of the path group is less than the threshold; If $B_t > B_r$, i.e. $A = 1$, the variance of the path group is greater than the threshold.

When the last data stream storage load balancing node in the saturated grid reaches the highest level threshold, the data stream storage node notifies its neighboring grid of an extended message G . After receiving the grid communication message, each node calculates whether it is within the extended data flow grid using the following formula

$$X_e = \begin{cases} X+1 \text{ and } X \% 2 = 0 \\ X-1 \text{ and } X \% 2 = 1 \end{cases} \quad (5)$$

$$Y_e = \begin{cases} Y+1 \text{ and } Y \% 2 = 0 \\ Y-1 \text{ and } Y \% 2 = 1 \end{cases} \quad (6)$$

Among them, X and Y are respectively used to describe the numbering of data flow grids in cloud computing.

Assuming that the set of subsequent nodes for calculating node L_n is described using $R_n = (L_{n+1}, L_{n+2}, \dots, L_{n+m})$, with a total of m nodes and different physical mappings of the same operator, and the set of corresponding data blocks for that node is described using $B_n = \{block_1, block_2, \dots, block_k\}$, $k \gg m$, and all $block$ are triplets, the formula is described as follows:

$$block_i = \langle L_n, dest, size \rangle \quad (7)$$

In the formula, i is the number of the data block; L_n is the node to which the data block belongs; $dest$ is the corresponding target node of the data block, which is $block_i.dest = L_j$, while $n < j \leq n+m$; $size$ is the number of tuples processed by the physiological data block, which reflects the size of the data block. $k \gg m$ ensures that all subsequent nodes and multiple data blocks correspond.

In order to minimize the variance of network link utilization and achieve more balanced load distribution, when selecting the optimal path group, priority is given to selecting path groups with link utilization variance less than the threshold. By selecting the threshold of the rerouting path group mentioned above and using the fitness function to balance the data load, the calculation formula is as follows

$$F^o = A \cdot (e^C)^{-A} \quad (8)$$

In the equation, the fitness function is represented as F^o , and the comprehensive value of the integrated link utilization and link utilization variance is represented as C .

III. RESULT ANALYSIS AND DISCUSSION

In order to verify the comprehensive effectiveness of the proposed data stream storage load balancing optimization method based on big data algorithms in cloud computing, a simulation needs to be conducted. In Windows 7 on a virtual machine, Visual C++6.0 and MATLAB 8.5 are used as development languages to build a simulation

environment for data stream storage load balancing optimization in cloud computing. Compare and simulate the proposed method with traditional methods. For some real-time monitoring data stream storage nodes in cloud computing, cloud computing center users need to obtain the required data stream in a timely manner. During the data stream storage process, the query latency of the data stream should be minimized as much as possible. Based on the advantage of data stream storage priority, the data stream query instructions issued by the grid center node can quickly obtain the information required by users. Figure 2 compares the internal query latency (ms) of data flow grid nodes in cloud computing, and the comparison results are shown in Figure 2.

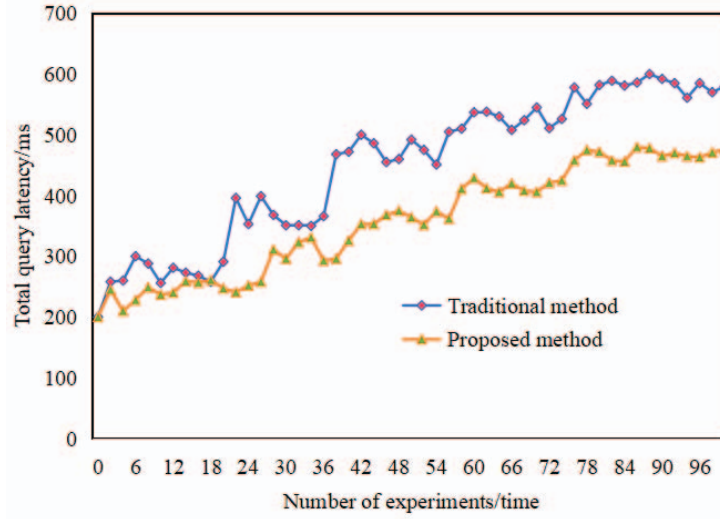


Figure 2: Different methods for data flow query response delay

From Figure 2, it can be seen that the load balancing optimization method for data stream storage in cloud computing based on big data algorithms exhibits strong real-time performance in the process of data stream query.

Compared with traditional methods, the delay in data stream query is greatly reduced. The comparison results show that the method proposed in this article effectively reduces data query delay.

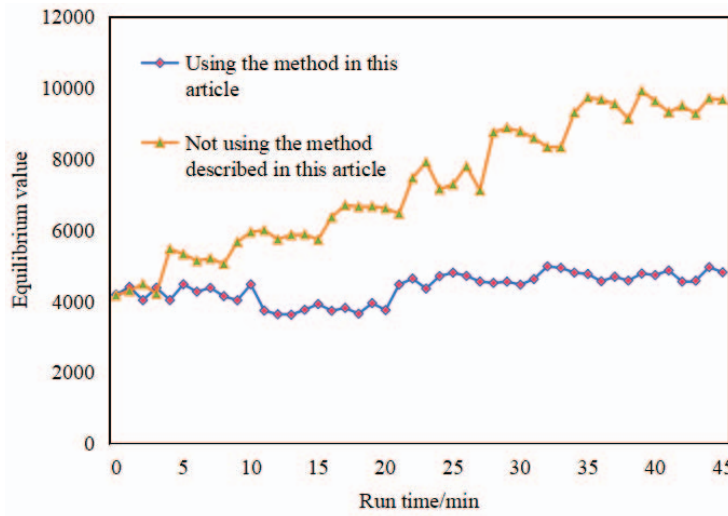


Figure 3 Timing chart of equilibrium values

Figure 3 shows the timing chart of node load balancing values in the system before and after using the proposed big

data algorithm based load balancing optimization method for data stream storage in cloud computing. In the

experiment, the system was run continuously for 45 minutes, and the equilibrium degree of the system that did not apply the method described in this article and the system that underwent equilibrium adjustment every 5 minutes was measured and recorded every 5 minutes. As shown in the figure, due to the continuous skewing of incoming data, systems without the application of this method gradually become imbalanced with increasing runtime; The equilibrium value of the system with the application of this method remains basically stable.

In order to further verify the dynamic load balancing performance of the proposed method, traditional methods were compared and the maximum load node average traffic and load balancing time were used as measurement

indicators for experimental testing. The average traffic of the maximum load node is a reflection of the congestion level of physiological data flow processing. The lower the value, the lower the congestion level is considered, and the higher the value, the higher the congestion level is considered. The load balancing method for data stream storage based on node classification requires searching for data stream storage points throughout the entire network environment, and ultimately storing the data stream in the most advantageous node for other nodes. It is generally believed that the lifespan of a node determines the lifespan of a network, so dynamic load balancing can be described by the similarity between the processing node and adjacent nodes.

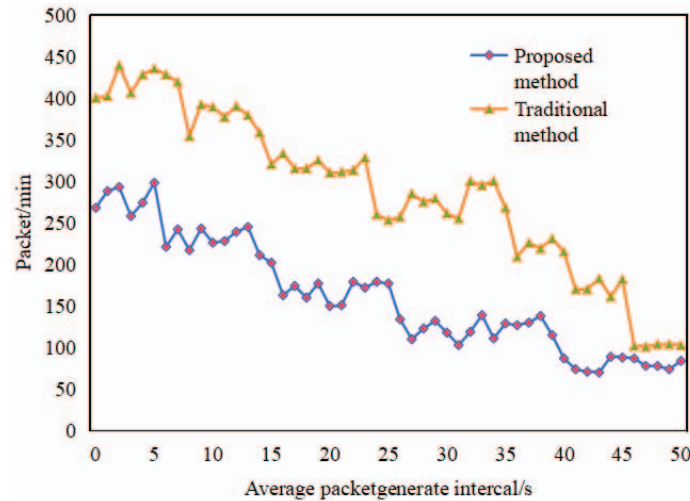


Figure 4 Comparison of average traffic of maximum load nodes

Analyzing Figure 4, it can be seen that the method proposed in this paper can effectively balance node load, making the maximum load node load smaller than

traditional methods. This indicates that the congestion level of the medical big data service system processed by the method proposed in this paper reaches the minimum.

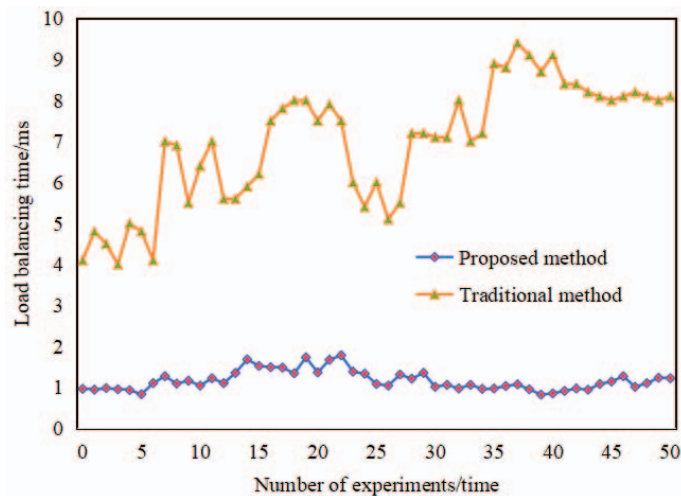


Figure 5 Comparison of load balancing time consumption

Analyzing Figure 5, it can be seen that the traditional method takes a long time and has significant fluctuations. In contrast, the load balancing method proposed in this paper takes a shorter time and is more stable. Considering the node load problem faced in the optimization process of data

stream storage load balancing, a multi threshold prediction mechanism and a dynamic virtual coordinate mechanism are adopted to allocate data stream storage nodes to the virtual grid. At the same time, multiple thresholds are set for different storage node resources, and the node coordinates

are adjusted through the selection of thresholds to prevent the problem of data stream storage load overload, which to some extent balances storage energy consumption.

IV. CONCLUSION

In the era of big data, more and more real-time applications require fast and accurate connection operations on large-scale real-time data streams, such as stock trading systems, online advertising analysis systems, etc. Due to the fast arrival rate and continuous generation of data streams, traditional connection systems cannot effectively meet the requirements of real-time systems mentioned above. Cloud computing technology and its derived cloud storage technology have certain significance in the design process of big data compatible storage systems. In response to the drawbacks of poor computing power and low load balancing ability of existing dynamic load balancing methods in physiological data stream network transmission, this paper proposes a simulation design method for optimizing load balancing of cloud computing data stream storage based on big data algorithms. This method analyzes the historical load data of storage nodes, uses big data algorithms to predict the future load situation of nodes, and dynamically allocates data to corresponding nodes based on this to achieve balanced distribution and optimization of load. The experimental results indicate that the proposed method balances the energy consumption of data stream storage to a certain extent by avoiding excessive node energy consumption and faster data stream query response. The method proposed in this article still has certain limitations, and in the future, it needs to be operated in larger scenes for a long time to observe its operation.

REFERENCES

- [1] Ye L. An optimization algorithm of data access storage in cloud computing based on distributed system. *Basic & clinical pharmacology & toxicology*, vol. 2019, no. 9, pp. 125, 2019.
- [2] Moori A, Barekatin B, Akbari M. LATOC: an enhanced load balancing algorithm based on hybrid AHP-TOPSIS and OPSO algorithms in cloud computing[J]. *Journal of supercomputing*, 2022(4):78.
- [3] Fuguang Y. Research on campus network cloud storage open platform based on cloud computing and big data technology. *Journal of Intelligent and Fuzzy Systems*, vol. 38, no. 1, pp. 1-9, 2019.
- [4] Barika M, Garg S, Chan A, et al. Scheduling Algorithms for Efficient Execution of Stream Workflow Applications in Multicloud Environments. *IEEE transactions on services computing*, vol. 2022, no. 2, pp. 15, 2022.
- [5] Zhang H, Zhang Z. Research on the Big Data Cloud Computing Based on the Network Data Mining. *Basic & clinical pharmacology & toxicology*, vol. 2019, no. 3, pp. 124, 2019.
- [6] Li X, Guo J. Research on Ship Data Big Data Parallel Scheduling Algorithm Based on Cloud Computing. *Journal of Coastal Research*, vol. 94, no. 1, pp. 535, 2019.
- [7] Jagdish M, Vilorio A, Vargas J, et al. Modeling software architecture design on data storage security in cloud computing environments. *Journal of Intelligent and Fuzzy Systems*, vol. 39, no. 6, pp. 1-8, 2020.
- [8] Yan S, He L, Seo J, et al. Concurrent Healthcare Data Processing and Storage Framework Using Deep-Learning in Distributed Cloud Computing Environment. *IEEE transactions on industrial informatics*, vol. 2021, no. 4, pp. 17, 2021.
- [9] Dogan D C, Altindis H. Storage and Communication Security in Cloud Computing Using a Homomorphic Encryption Scheme Based Weil Pairing. *Elektronika ir Elektrotechnika*, vol. 26, no. 1, pp. 78-83, 2020.
- [10] Xu H. Classification and Storage Method of Marine Multi-source Transmission Data under Cloud Computing. *Journal of Coastal Research*, vol. 115, no. 1, pp. 84, 2020.